CineMatch

AI-Powered Personalized Movie, TV, Anime & K-Drama Recommendation System

PROJECT REPORT

Submitted by	Ashish Kumar Prusty
Domain	Artificial Intelligence & Machine Learning
Category	Full-Stack AI Web Application
Live Deployment	https://cinematch-0tab.onrender.com
Repository	github.com/ASHISH8652/-AI-Movie-Recommendation-System
Location	Odisha, India

Table of Contents

1. Abstract

CineMatch is a full-stack, AI-powered recommendation system that delivers personalized movie, television, anime, and Korean drama suggestions to individual users. Unlike conventional recommendation demos that operate on static, pre-loaded sample datasets, CineMatch integrates directly with The Movie Database (TMDB) API to source live, continuously updated content spanning new releases, decades-old classics, and titles across more than twenty-five languages, including major regional Indian languages such as Odia, Tamil, Telugu, Bengali, Marathi, Gujarati, Kannada, and Malayalam.

At the core of the system is a per-user Gradient Boosting Classifier, trained on a feature set that blends explicit feedback (thumbs up/down, one-to-five star ratings), implicit behavioural signals (titles a user opens), and TMDB's own public content signals (genre, popularity, audience rating, release era, spoken language, and media type). Each user's model is trained, stored, and served independently, ensuring that one person's viewing taste never influences another's recommendations. The model retrains automatically as feedback accumulates, progressively sharpening its predictions without requiring any manual intervention.

The system is secured using a passwordless, email-based one-time-password (OTP) authentication flow, with session management handled through cryptographically signed cookies. The backend is implemented in Python using the FastAPI framework, with SQLAlchemy providing a database abstraction layer that allows the application to run on SQLite in local development and PostgreSQL in production without any code changes. The complete application is deployed on Render as a live, publicly accessible web service, backed by a managed PostgreSQL database, demonstrating an end-to-end pipeline from data ingestion through model training to production deployment.

This report documents the motivation, design, architecture, machine learning methodology, security model, and deployment process behind CineMatch, along with the practical engineering challenges encountered during real-world cloud deployment and their resolutions.

2. Introduction

The volume of movie, television, and streaming content available to audiences today has grown far beyond what any individual can manually browse. Viewers routinely spend more time searching for something to watch than actually watching it — a phenomenon widely referred to as “decision fatigue.” Generic, one-size-fits-all trending lists fail to account for an individual's specific taste, mood, language preference, or the type of content they are looking for at a given moment, whether that is a light comedy, a Korean drama, an anime series, or a classic film from decades past.

CineMatch was built to address this gap directly: a recommendation system that combines real-time content discovery with genuine machine learning personalization, wrapped in a production-grade web application that is authenticated, deployed, and usable by real people — not a notebook-only proof of concept.

3. Problem Statement & Objectives

Problem Statement: Existing free movie-discovery tools are largely static and non-personalized; they surface the same trending or top-rated titles to every user regardless of individual taste, mood, language preference, or content-type interest (movies vs. series vs. anime vs. K-drama).

Objectives:

- Provide real-time, always-current content data rather than a fixed local dataset.
- Support multiple content categories — movies, TV series, anime, and Korean dramas — under one interface.
- Support mood-based and genre-based discovery (comedy, action, horror, romance, thriller, and more).
- Support content in 25+ languages, with strong coverage of Indian regional languages.
- Learn each individual user's taste from their own feedback, using a genuinely trained ML model rather than a fixed scoring formula.
- Secure user accounts without requiring users to create or remember passwords.
- Deploy the system as a real, publicly reachable web application rather than a local-only script.

4. Literature Survey / Existing Systems

Commercial platforms such as Netflix and Amazon Prime Video employ large-scale collaborative filtering and deep learning recommendation pipelines, but these are proprietary, closed systems trained on billions of interactions across a fixed content catalogue that the platform itself owns and licenses. Open-source and academic recommendation projects commonly rely on the MovieLens dataset, which is static, English-centric, and does not reflect current release schedules, regional content, or multi-format media such as anime and K-dramas.

CineMatch differs from both categories: it uses a live, licensed third-party data source (TMDB) for content, rather than a static dataset, and it implements a lightweight, genuinely trained per-user classifier rather than relying purely on collaborative filtering across a large user base — an approach that remains effective even for a system with a small or newly onboarded user base, since each model only needs that one user's own feedback to begin personalizing.

5. Proposed System

CineMatch is architected as a single FastAPI backend service that (1) authenticates users via email OTP, (2) proxies and enriches live TMDB data on demand, (3) records every user interaction as a labelled training example, and (4) trains and serves a personalized scikit-learn classifier per user. The frontend is a single-page application served directly by the backend, communicating over a JSON REST API secured by session cookies.

6. System Architecture

Layer	Component	Responsibility
Presentation	static/index.html (HTML/CSS/JS)	Login UI, mood/genre/era filters, results grid, rating controls, weekly-vibe dash
API	app.py (FastAPI)	Route handling, session auth guard, request orchestration
Auth	auth.py	OTP generation/hashing, Resend email delivery, signed session tokens
Data Access	db.py (SQLAlchemy)	User, OTP, and feedback persistence — SQLite locally, PostgreSQL in produ
ML	recommender.py	Feature engineering, per-user model training, scoring
External Data	tmdb_client.py	TMDB API integration — discover, search, trending, details

Request flow: the browser calls the FastAPI backend exclusively; the TMDB API key and email-provider API key never reach client-side code. This keeps all third-party credentials server-side, which is both a security requirement and a deployment necessity, since browser-embedded keys would be extractable by any user.

7. Technology Stack

Layer	Technology
Backend Framework	FastAPI (Python)
ASGI Server	Uvicorn
Machine Learning	scikit-learn — GradientBoostingClassifier
ORM / Database Layer	SQLAlchemy 2.x
Local Database	SQLite
Production Database	PostgreSQL (managed, via Render)
Authentication	Email OTP + itsdangerous signed session cookies
Email Delivery	Resend HTTP API
External Content Data	TMDB (The Movie Database) API
Frontend	HTML5, CSS3, vanilla JavaScript (fetch API)
Hosting / Deployment	Render (web service + managed Postgres, via render.yaml Blueprint)
Version Control	Git & GitHub

8. Machine Learning Model

8.1 Algorithm

CineMatch uses a **Gradient Boosting Classifier** (scikit-learn), trained independently for each user. Gradient boosting was chosen over deep learning approaches because it performs well on small, tabular, mixed-signal datasets — exactly the shape of a single user's early feedback history — without requiring the large volume of training examples a neural network would need.

8.2 Feature Engineering

- Genre — multi-hot encoded across a fixed 27-genre TMDB vocabulary (movie and TV genres unioned).
- Original language — one-hot encoded across a 26-language vocabulary with an “other” fallback bucket.
- TMDB audience rating — normalized to a 0–1 scale.
- Popularity — log-scaled and capped to reduce the influence of viral outliers.
- Release year — normalized across a 1950–2026 range.
- Media type — binary flag distinguishing movies from TV/series content.

8.3 Label Construction from Feedback Signals

Signal	Label	Sample Weight
■■ Like	Positive (1)	1.0
■■ Dislike	Negative (0)	1.0
★ Rating ≥ 4	Positive (1)	1.0
★ Rating ≤ 2	Negative (0)	1.0
★ Rating = 3 (neutral)	Excluded from training	—
Click / opened title	Weak positive (1)	0.35

8.4 Training Trigger & Lifecycle

- A user's very first searches use a content-popularity baseline score (TMDB rating and popularity only) so the app is useful before any feedback exists.
- Once a user has accumulated at least 8 labelled examples containing both positive and negative signals, their personal model trains for the first time.
- The model automatically retrains every 5 additional feedback events.
- Users may also force an immediate retrain from the interface.
- Trained models are persisted per-user (via joblib) alongside metadata describing sample count, training timestamp, and the top genre features driving that user's predictions.

8.5 Verification

The personalization pipeline was verified with a controlled experiment: two synthetic user profiles were created — one that liked science-fiction/action titles and disliked documentaries, and a second with the exact opposite preferences. After training, both models were asked to score the same two held-out titles. The two users' models produced opposite rankings on identical content (100/100 vs. 8/100 personalized score on the same sci-fi title, and the inverse on the documentary), confirming that personalization is genuinely user-specific and not a shared or averaged signal.

9. Database Design

Table	Key Columns	Purpose
users	id, email, created_at	One row per authenticated account
otp_codes	id, email, code_hash, expires_at, consumed	Hashed, time-limited OTP codes for login verification
feedback	id, user_id (FK), tmdb_id, media_type, genre_ids, liked, rating_value, feedback_context, created_at	Every explicit and implicit signal, used to build each user's ML training data

The schema is defined once using SQLAlchemy's declarative ORM and runs unmodified against both SQLite (local development) and PostgreSQL (production), fulfilling the goal of an identical codebase across environments.

10. Authentication & Security

- **Passwordless login:** users authenticate with a 6-digit OTP emailed to their address; no password is ever created or stored.
- **OTP hashing:** codes are stored only as SHA-256 hashes with a 10-minute expiry, never in plaintext.
- **Session tokens:** issued as cryptographically signed cookies (itsdangerous), avoiding the need for a server-side session store.
- **Credential isolation:** the TMDB API key, email-provider API key, and session-signing secret are all read from server-side environment variables and are never exposed to client-side JavaScript.
- **Per-user data isolation:** every feedback record and trained model is scoped to a single user ID; there is no cross-user data leakage in either the database queries or the model file paths.

11. Module-Wise Implementation

Module	Description
app.py	FastAPI application: route definitions, authentication dependency injection, TMDB request orchestration (
auth.py	OTP generation and SHA-256 hashing, Resend-based email delivery over HTTPS, and signed session-to

db.py	SQLAlchemy engine/session setup (environment-driven SQLite/PostgreSQL switch) and ORM models fo
recommender.py	Feature vector construction, per-user Gradient Boosting model training, persistence, and scoring, includin
tmdb_client.py	A thin wrapper around the TMDB v3 REST API for discover, search, trending, details, and genre-list endp
static/index.html	The complete single-page frontend: login/OTP UI, mood and filter controls, results grid, detail modal with

12. Deployment Architecture

CineMatch is deployed on Render using an infrastructure-as-code Blueprint (`render.yaml`), which provisions a Python web service and a managed free-tier PostgreSQL database in a single step, and wires the database connection string to the web service automatically.

Live deployment: **https://cinematch-0tab.onrender.com**

12.1 Deployment Challenges Resolved

- **Native build failure (pydantic-core / Rust):** the platform's default Python version lacked a prebuilt wheel for the pinned pydantic-core version, forcing a source build that failed inside Render's read-only build cache. Resolved by relaxing dependency version pins so pip resolves builds with available prebuilt wheels.
- **Outbound SMTP blocked:** direct Gmail SMTP delivery failed in production (connection timeouts) because free-tier cloud hosts commonly block outbound SMTP ports to prevent abuse. Resolved by switching OTP delivery to the Resend HTTP email API, which operates over standard HTTPS and is not subject to SMTP port restrictions.
- **Secret exposure remediation:** an early commit accidentally included a local `.env` file. This was resolved by rotating all exposed credentials and performing a clean git history rebuild before any further deployment.

13. Testing

Test	Method	Result
Auth gate enforcement	Anonymous request to a protected endpoint	401 Unauthorized, as expected
Invalid OTP rejection	Verify endpoint called with an incorrect code	Rejected with a clear error, session not issued
Per-user model isolation	Two synthetic users trained on opposite preferences	Opposite personalized scores confirmed on identical titles
End-to-end local run	Full server boot, static file serving, and API routes	All endpoints responded correctly
Production build	Live Render deployment build and boot	Build succeeded; service reported live and serving 200 OK

14. Results & Discussion

The deployed system successfully authenticates users via emailed OTP, retrieves live content from TMDB across all supported categories and languages, and produces measurably different personalized rankings for users with different feedback histories. The system requires no manual dataset maintenance, since all content data is sourced live at request time, and the model-training pipeline operates without any manual intervention beyond the user's own rating activity.

15. Advantages & Limitations

Advantages

- Always-current content — no stale, manually curated dataset.
- Genuine per-user personalization rather than a static heuristic.
- No password to manage or leak — OTP-only authentication.
- Identical codebase runs locally (SQLite) and in production (PostgreSQL).
- Broad content coverage: movies, TV, anime, and K-dramas across 25+ languages.

Limitations

- Cold-start: a brand-new user sees only the popularity-based baseline until they accumulate feedback.
- Free-tier hosting constraints: the web service's local disk is not persistent across redeploys, which can require a one-click model retrain after a redeploy (user data itself remains safe in PostgreSQL).
- Email delivery to third-party users currently requires a verified sending domain on the email provider for unrestricted recipient delivery.

16. Future Scope

- Verified sending domain to allow unrestricted account creation for any user's email address.
- Cross-user collaborative filtering layered on top of the existing per-user model.
- Watchlist / save-for-later functionality.
- “Why this was recommended” explanation surfaced per title.
- Social login (Google/GitHub) as an alternative to email OTP.
- Progressive Web App (PWA) packaging for a native-like mobile experience.

17. Conclusion

CineMatch demonstrates a complete, production-oriented approach to building a personalized recommendation system: live third-party data integration, genuine per-user machine learning rather than a static scoring formula, passwordless authentication, and a real cloud deployment reachable by any user. The project's engineering scope extended beyond model design into the practical realities of secure credential handling, environment-portable database design, and diagnosing and resolving real production deployment failures — experience directly representative of professional full-stack AI application development.

18. References

- The Movie Database (TMDB) API Documentation — https://developer.themoviedb.org/

- FastAPI Documentation — https://fastapi.tiangolo.com/
- scikit-learn: Gradient Boosting — https://scikit-learn.org/stable/modules/ensemble.html#gradient-boosting
- SQLAlchemy Documentation — https://docs.sqlalchemy.org/
- Resend API Documentation — https://resend.com/docs
- Render Deployment Documentation — https://render.com/docs

Prepared by Ashish Kumar Prusty — Odisha, India